Modeling and Structural Analysis of Chaotianmen Bridge Using a Mixed 3D Truss–Frame Direct Stiffness Method

Chisung Kim

School of Civil and Environmental Engineering, Georgia Institute of Technology

Abstract

ABC

Keywords: Direct Stiffness Method; 3D truss; 3D frame; mixed-element analysis; bridge modeling; Chaotianmen Bridge

1. Introduction

This project develops a Python-based direct stiffness method (DSM) program for mixed structural systems consisting of 3D truss, 3D frame, and cable elements. The program is designed so that structural models can be defined entirely through input files without modifying the source code, and the main workflow is executed through `main.ipynb`. In addition to the implementation of the solver, the project includes verification against hand-solvable benchmark problems and an application study on a real bridge structure. The overall scope and deliverables of the project were developed in accordance with the final project specification provided in the course repository (`https://github.com/Bruun-Automation-Research-Lab/CEE6501/blob/main/Assignments/A11_FinalProject.md`).

The selected case study is the Chaotianmen Bridge in Chongqing, China, a long-span steel road–rail arch bridge. This bridge was chosen because it provides a meaningful mixed-element modeling problem: the global structural system contains continuous chord and arch members that are naturally idealized as frame elements, together with verticals, diagonals, and bracing members that are more appropriately represented as truss elements. In addition, its geometry, photographs, and publicly available bridge information provide sufficient background for a rational simplified structural model.

The final goal of the project is not only to solve the baseline structural response, but also to investigate the structural behavior under a range of loading conditions and modeling assumptions. In addition to the baseline case including self-weight, road traffic load, and rail load, a series of what-if scenarios is considered to examine the sensitivity of the bridge response to temperature change, support settlement, fabrication error, member removal, and end-release assumptions. These scenarios are used to evaluate how both loading conditions and structural idealization influence the global behavior of the bridge.

2. Program Overview and User Guide

2.1 Repository Structure and Helper Modules

The GitHub repository for this project is available at the following link:

`https://github.com/kimchi-chisung/CEE6501_MSA_final_course_project`

The repository is structured so that model generation, structural analysis, validation reference files, and result storage are clearly separated. This organization makes the workflow easier to reproduce and simplifies debugging, validation, and later extension of the code. The main top-level structure of the repository is shown below.

```
helpers/
inputs/
outputs/
presentation/
report/
validation_ref/
input_generation.ipynb
main.ipynb
requirements.txt
```

The `helpers/` folder contains the main Python modules used by the solver, while the `inputs/` folder stores the structural input files for both validation examples and final case studies. Generated analysis results, including spreadsheets, saved figures, and interactive plots, are written to the `outputs/` folder. The `presentation/` and `report/` folders contain the presentation materials and written report files, respectively. The `validation_ref/` folder stores the handwritten reference solutions used in the validation study, so that the computed results from the code can be compared directly against the original hand-calculated benchmark values.

The project workflow is organized around two Jupyter notebooks. The primary analysis notebook, `main.ipynb`, is used to run the structural analysis, review the computed response, and generate tables and figures. A separate notebook, `input_generation.ipynb`, is used to create or modify larger structural input files, particularly for the Chaotianmen Bridge model.

Within the `helpers/` folder, the source code is divided into several helper modules according to the main stages of the analysis workflow. The preprocessing module reads the input model and organizes the nodal, element, support, and loading information into the internal format used by the solver. The element module contains the truss and frame element formulations, including local stiffness matrices, coordinate transformations, and element-level response quantities. The assembly module constructs the global stiffness matrix and global equivalent load vectors from the element

contributions, while the partitioning module separates the system into free and restrained degrees of freedom for solution.

The solver module computes the unknown displacement quantities and recovers the restrained reactions. The fixed-end-force module evaluates member-level contributions associated with distributed loads, point loads, temperature effects, and fabrication error. The release module handles frame end releases so that moment-release conditions can be included consistently within the frame formulation. Finally, the postprocessing module organizes the solved results into tables, summaries, and plots for interpretation and export. A more detailed summary of the repository structure and the principal functions contained in each helper module is provided in Appendix A.

2.2 Main Analysis Workflow in `main.ipynb`

The complete structural analysis workflow is executed from `main.ipynb`. In typical use, the user first selects a structural input file by setting the variable `model_path` to one of the JSON models stored in the `inputs/` directory. The notebook then reads the selected model, extracts the structural data, assembles the global stiffness system, solves for nodal displacements and reactions, computes element-level response quantities, and finally exports tables and plots. Thus, the intended workflow is straightforward: choose an input file, run the notebook cells, and review the generated numerical and graphical outputs.

A typical model-selection block in `main.ipynb` is shown below.

```
# Validation Truss
# model_path = "inputs/validation_truss.json"

# Validation Frame
# model_path = "inputs/validation_frame_original.json"
# model_path = "inputs/validation_frame_moment_release.json"
# model_path = "inputs/validation_frame_prescribed_displacement.json"
# model_path = "inputs/validation_frame_temperature_loads.json"
# model_path = "inputs/validation_frame_fabrication_error.json"

# Final Structure
model_path = "inputs/chaotianmen_bridge.json"

model = load_model(model_path)
model_name = Path(model_path).stem
```

After the model is loaded, the notebook reads the principal fields from the JSON file, including the nodes, elements, supports, nodal loads, prescribed displacements, member loads, temperature loads,

fabrication errors, and releases. These quantities are then used to construct the global load vector, the prescribed displacement vector, the restrained degree-of-freedom list, and the element-level stiffness and equivalent load contributions required for the mixed truss–frame analysis.

The execution sequence in `main.ipynb` can be summarized as follows:

1. Import the required helper modules for preprocessing, element formulation, assembly, partitioning, solving, fixed-end force evaluation, release handling, and postprocessing.
2. Select the input model by setting `model_path`.
3. Read the JSON file using `load_model`.
4. Build the global load and displacement vectors and identify restrained DOFs.
5. Generate element stiffness matrices, transformation matrices, and local fixed-end force vectors.
6. Assemble the global stiffness matrix and equivalent load vector.
7. Partition the system into free and restrained degrees of freedom.
8. Solve for unknown displacements and compute support reactions.
9. Recover complete global vectors and compute element-level results.
10. Generate tables and plots for display and/or saving.

At the end of the analysis workflow, `main.ipynb` uses two high-level postprocessing functions: `build_result_tables(...)` and `plot_results(...)`. These functions serve as the main interface for reviewing and exporting the final results.

The function `build_result_tables(...)` is responsible for collecting the numerical results into structured tables. It gathers element-level results, displacement and reaction vectors, element direction cosines, fixed-end force data, and other intermediate or summary quantities, and organizes them into spreadsheet sheets that can later be reviewed in Excel. Depending on the structural model, the main sheets include `truss`, `frame`, and `cable`, followed by supplementary sheets such as `element_cosines`, `Qf_local`, `u_f`, `F_r`, `u_global_complete`, and `f_global_complete`. The function can either display the tables inside the notebook or save them directly to the `outputs/` directory as an `.xlsx` file.

A typical table-generation block is shown below.

```
tables = build_result_tables(
    elements=elements,
    element_lengths=element_lengths,
    results_truss=results_truss,
```

```
    results_cable=results_cable,
    results_frame=results_frame,
    display_tables=False,
    save_tables=True,
    save_dir="outputs",
    model_path=model_path,
    nodes=nodes,
    supports=supports,
    Qf_debug=Qf_debug,
    u_f=u_f,
    F_r=F_r,
    u_global_complete=u_global_complete,
    f_global_complete=f_global_complete,
    F_global=F_global,
    F_fef_global=F_fef_global,
    releases=releases,
)
```

The function `plot_results(...)` is responsible for generating graphical output. It receives the solved structural model, including the nodal coordinates, element definitions, global displacement vector, and element-level result dictionaries, and then produces undeformed geometry plots, deformed-shape plots, and response plots for axial force, shear force, and bending moment. The user can control whether the plots are displayed interactively in the notebook, saved to disk, or both. For the Chaotianmen Bridge model, the option `split_by_y_plane_2d=True` is especially useful because it separates the two longitudinal planes in the 2D plots and avoids visual overlap of members located at different y-coordinates.

A typical plotting block is shown below.

```
plot_results(
    nodes=nodes,
    elements=elements,
    u_global_complete=u_global_complete,
    results_truss=results_truss,
    results_cable=results_cable,
    results_frame=results_frame,
    supports=supports,
    nodal_loads=nodal_loads,
    display_3d=True,
    save_3d=True,
```

```
    display_2d=True,
    save_2d=True,
    model_path=model_path,
    split_by_y_plane_2d=True,
)
```

Together, these two functions define the final postprocessing stage of `main.ipynb`. The first emphasizes numerical organization, export, and reproducibility of the analysis results, while the second emphasizes visual interpretation of the structural response.

2.3 Input File Format

Each structural model is defined through a JSON input file. The input format is organized by structural role, so that nodes, elements, supports, loads, prescribed displacements, temperature loads, fabrication errors, and releases can be specified independently. If a particular feature is not used in a given model, the corresponding field may simply be left as an empty dictionary, {}.

Nodes The `nodes` field defines the nodal coordinates. Each node is identified by an ID, and its value is given as a three-component coordinate vector:

```
"nodes": {
  "1": [0.0, 0.0, 0.0],
  "2": [0.0, 10.0, 0.0],
  "3": [8.0, 10.0, 0.0]
}
```

Even for planar examples, the coordinates are written in 3D form $[x, y, z]$ because the solver is implemented as a 3D program.

Elements The `elements` field defines the element type, connectivity, and material/section properties. A typical frame element is written as

```
"elements": {
  "1": {
    "type": "3D_frame",
    "nodes": [1, 2],
    "E": 200000000.0,
    "A": 0.00474,
    "I": 0.0000222,
```

```
    "J": 0.00001
  }
}
```

For `3D_frame` elements, the input includes `A`, `I`, and `J`. For `3D_truss` and `3D_cable` elements, only the axial area `A` is required.

Supports The `supports` field specifies restrained degrees of freedom. Each support node is associated with a list of restrained DOFs:

```
"supports": {
  "1": ["ux", "uy", "uz", "rx", "ry", "rz"],
  "2": ["uz", "rx", "ry"]
}
```

In this example, node 1 is fully fixed, while node 2 is restrained only in `uz`, `rx`, and `ry`.

Nodal Loads The `nodal_loads` field defines concentrated nodal forces and moments. Each value is written as a six-component vector:

```
"nodal_loads": {
  "31": [0.0, 0.0, -200.0, 0.0, 0.0, 0.0]
}
```

The six entries correspond to $[F_x, F_y, F_z, M_x, M_y, M_z]$.

Member Loads The `member_loads` field is used to define loads applied directly to individual members. In the current implementation, this field can represent multiple loading types, including uniformly distributed loads, concentrated point loads, and linearly varying distributed loads. Depending on the load type, the input may include load magnitude, direction, and application location parameters. In this input format, the load direction is specified in the global coordinate system; the corresponding load components are later transformed internally into the local element coordinate system during fixed-end-force evaluation. A simple example is

```
"member_loads": {
  "1": {"wx": 24.0},
  "2": {"Py": -75.0, "aP": 4.0}
}
```

where element 1 carries a distributed load and element 2 carries a point load applied at a specified location along the member.

Releases The `releases` field is used to assign frame end releases. Releases are specified element by element, and the released degree of freedom is identified by the element end (`i` or `j`) and the local rotational DOF. For example,

```
"releases": {
  "1": ["j_ry"],
  "2": ["i_rx", "j_rz"]
}
```

More generally, the release definition may include end-specific rotational releases such as `i_rx`, `i_ry`, `i_rz`, `j_rx`, `j_ry`, and `j_rz`, depending on the intended connection idealization.

Temperature Loads The `temperature_loads` field is used to define element-level thermal loading. The simplest case is a uniform temperature change, specified through `alpha` and `T_avg`. The current format also allows an additional gradient-type term through `dTz` and `dz`. Here, `dTz` represents the temperature difference across the section in the local z-direction, and `dz` denotes the corresponding section depth measured along that same local direction. For example,

The `temperature_loads` field is used to define element-level thermal loading. The simplest case is a uniform temperature change, specified through `alpha` and `T_avg`. The current format also allows an additional gradient-type term through `dTz` and `dz`. For example,

```
"temperature_loads": {
  "1": {
    "alpha": 12e-6,
    "T_avg": 10.0
  },
  "2": {
    "alpha": 12e-6,
    "T_avg": 10.0,
    "dTz": -10.0,
    "dz": 0.3
  }
}
```

Fabrication Errors The `fabrication_errors` field is used to define member fabrication error in the form of axial length error. In the current implementation, this is specified element by element using the parameter `e_a`. For example,

```
"fabrication_errors": {
```

```
  "1": {"e_a": -0.003},
  "2": {"e_a": -0.003}
}
```

At present, fabrication error is modeled only as member length error, consistent with the simplification adopted in this study.

2.4 Generated Output Files

The program generates both tabular and graphical outputs. The tabular outputs are written to an Excel workbook in the `outputs/` directory, while the graphical outputs are saved as image files and, when enabled, as a Plotly-based interactive 3D HTML file. Together, these outputs provide a reproducible numerical record of the analysis and a visual summary of the structural response.

Excel Workbook Output The main numerical output is stored in an `.xlsx` workbook generated by `build_result_tables(...)`. The workbook contains both element-level response tables and supplementary system-level data used for checking, interpretation, and debugging.

The principal sheets are:

- `truss`: results for truss members,
- `frame`: results for frame members,
- `cable`: results for cable members.

These sheets contain the basic element information needed to interpret the results, such as the element ID, end-node IDs, member length, and related response quantities. Depending on the element type, the sheets also store quantities such as axial force, stress, local displacement components, and local end-force components. As a result, these sheets serve as the primary tables for reviewing member-by-member structural response.

In addition, the workbook includes supplementary sheets such as:

- `element_cosines`: element direction cosines and geometric information,
- `Qf_local`: local fixed-end force vectors associated with member loads, temperature loads, or fabrication errors,
- `u_f`: solved displacement vector at the free degrees of freedom,
- `F_r`: support reaction vector at the restrained degrees of freedom,
- `u_global_complete`: reconstructed full global displacement vector,

- `f_global_complete`: reconstructed full global force vector.

Depending on the analysis case and enabled options, the workbook may also include summary-style outputs derived from the solved results, such as displacement summaries, reaction summaries, member-force summaries, equilibrium checks, and release-related bookkeeping.

A typical table-generation block is shown below.

```
tables = build_result_tables(
    ...,
    display_tables=False,
    save_tables=True,
    save_dir="outputs",
    ...
)
```

Here, `display_tables` controls whether the tables are shown directly in the notebook, while `save_tables` controls whether they are exported to the Excel workbook.

Graphical Output The graphical outputs are generated to describe both the structural model and the computed response. Geometry figures show the undeformed structural layout, while element-type figures distinguish frame, truss, and cable members using different colors or line styles. These outputs are useful for confirming that the structural model has been assembled correctly and that each component has been assigned the intended element type.

The response figures include deformed-shape plots, axial-force plots, shear-force plots, and bending-moment plots. The deformed-shape figures compare the undeformed and deformed configurations of the structure, with a scale factor shown in the title or legend so that small displacements can be visualized clearly. The axial-force, shear-force, and bending-moment figures use color to indicate the sign and relative magnitude of the corresponding response quantity while preserving the structural geometry.

For the final bridge model, these graphical outputs are generated in several complementary forms. The overall 2D plots provide a clear side-elevation summary of the full bridge response, while the 3D plots show the complete spatial model and make it easier to check the global layout in three dimensions. In addition, split 2D views for $Y < 0$ and $Y > 0$ are generated because the bridge model consists of two longitudinal planes; separating these planes avoids overlap and makes each side elevation easier to read.

A typical plotting block is shown below.

```
plot_results(
```

```
    ...,
    display_3d=True,
    save_3d=True,
    display_2d=True,
    save_2d=True,
    split_by_y_plane_2d=True,
    ...
)
```

Here, `display_3d` and `display_2d` determine whether the plots appear during notebook execution, while `save_3d` and `save_2d` determine whether the figures are written to the output directory. The option `split_by_y_plane_2d=True` enables separate side-elevation plots for the two longitudinal planes.

In addition to the static image files, the program can also generate an interactive 3D HTML visualization using Plotly. This output provides the same structural model in a form that can be rotated, zoomed, and inspected more freely than a static figure, making it a useful supplementary tool for reviewing the full bridge geometry and overall response.

Overall, the generated output set is designed to provide both detailed numerical results and readable visual summaries. The Excel workbook stores the tabulated response quantities needed for verification and reporting, while the figures and interactive Plotly visualization provide direct graphical access to the model geometry, deformation pattern, and internal-force distribution.

3. Formulation and Modeling Assumptions

3.1 Linear Structural Idealization

The analysis is carried out within a linear-elastic framework. Force and displacement are therefore assumed to be linearly related, and the results from different load cases can be superposed. This is the basis for the later what-if studies, where self-weight, traffic load, temperature load, fabrication error, and support settlement are examined within the same model.

Geometric nonlinearity is not included. Member geometry is taken from the undeformed configuration, and stiffness changes caused by deformation are neglected. Material nonlinearity is also excluded, so all members are assumed to remain elastic throughout the analysis.

3.2 Member-Level Kinematic Assumptions

Truss and cable elements are treated as axial-force members. Their bending, shear, and torsional resistance are neglected. Frame elements are used for members that are intended to carry both axial

force and bending.

The frame formulation is simplified in two ways. First, the same bending stiffness is used in the two principal bending directions, so the section is treated as having identical flexural rigidity in the local y- and z-bending directions. This avoids the need for a more general 3D frame treatment with distinct principal inertias and explicit roll-angle definition. Second, the local element coordinate system is defined from the undeformed member geometry and kept fixed during the analysis. In other words, the member response is evaluated with respect to local directions established from the original configuration, consistent with the small-deformation assumption of the linear model.

The coordinate transformation approach used for frame elements follows the same geometric idea used for the 3D truss formulation. A member axis is first defined from the end-node geometry, and a consistent local coordinate system is then constructed for element-level calculations and local-force recovery. In this sense, the geometric transformation-matrix approach used for 3D truss elements is extended here to the frame formulation, while the frame element itself retains the additional bending-related degrees of freedom required for flexural response.

3.3 Modeling Simplifications for the Present Study

To keep the project within the intended course scope, several additional simplifications were adopted. Large-displacement effects, P–Δ effects, and large rotations are not considered. Connection flexibility is not modeled explicitly unless it is introduced through an idealized frame end release. Fabrication error is represented only as axial length error, not as initial curvature or other geometric imperfection.

These assumptions define the scope of the present analysis and should be considered when interpreting the results for the final bridge model. The reported results therefore represent linear-elastic response predictions obtained from a simplified structural model.

4. Validation

The developed solver was validated against benchmark problems with available hand solutions from earlier course assignments. Since the implemented code is a 3D solver, the validation was performed by embedding two-dimensional problems into three-dimensional space and restraining the out-of-plane degrees of freedom as needed. This made it possible to verify the 3D implementation while still comparing the computed response with analytically tractable planar problems.

Two separate validation groups were used: a truss validation problem and a frame validation problem. To further confirm that loading directions, restraint definitions, and degree-of-freedom handling were implemented consistently, the two validation models were embedded in different planes. The truss example was placed in the x-z plane, while the frame example was placed in the x-y plane. This provided an additional check that the solver was not tied to a single preferred orientation and that the 3D formulation behaved correctly in different coordinate directions.

4.1 Truss Validation

The truss validation model was defined in `inputs/validation_truss.json`, and the reference hand solution was taken from `validation_ref/Kim_A5_written.pdf`. The original planar problem was embedded into the x-z plane in the present 3D solver by restraining the out-of-plane degrees of freedom. This made it possible to check the 3D implementation against a problem with a known hand solution.

Figure 1 shows the original truss geometry and loading used in the handwritten solution, and Figure 2 shows the corresponding computed axial-force distribution from the 3D solver. The comparison focused on support reactions and member axial forces, including the tension/compression state of each member.

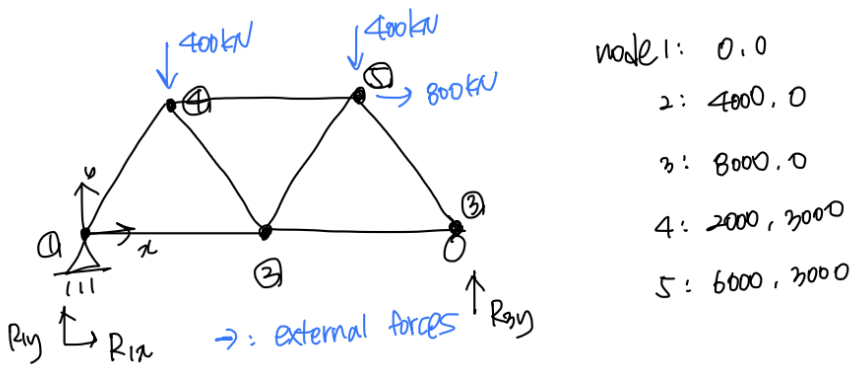

Figure 1: Handwritten truss layout, nodal coordinates, support conditions, and applied loading used for the validation problem.

Table 1 compares the support reactions from the hand calculation and the developed solver. Table 2 compares the axial forces of the truss members. Because the handwritten solution and the computed output may use different element numbering, the axial-force comparison is organized by member connectivity rather than by element ID.

Table 1: Comparison of support reactions for the truss validation problem.

Quantity	Hand calculation (kN)	Computed result (kN)
Node 1, R_x	-800.0	-800.0
Node 1, R_z	100.0	100.0
Node 3, R_z	700.0	700.0

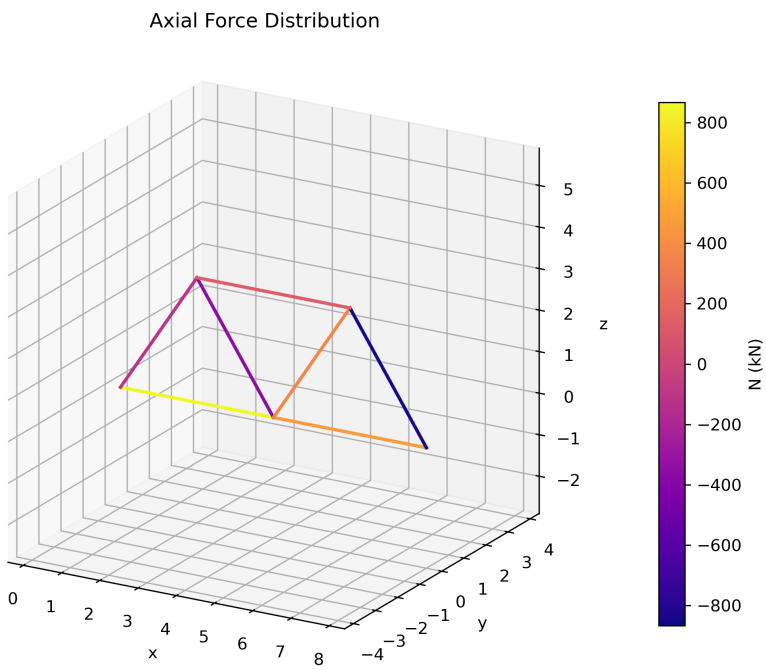

Figure 2: Computed 3D axial-force distribution for the validation truss model embedded in the x-z plane.

Table 2: Comparison of axial member forces for the truss validation problem.

Element	Member connectivity	Hand calculation (kN)	Computed result (kN)	T/C
1	$1 \to 2$	866.667	866.667	T
2	$2 \to 3$	466.667	466.667	T
3	$4 \to 5$	133.333	133.333	T
4	$1 \to 4$	-120.185	-120.185	C
5	$4 \to 2$	-360.555	-360.555	C
6	$2 \to 5$	360.555	360.555	T
7	$5 \to 3$	-841.295	-841.295	C

4.2 Frame Validation

The original frame validation model was defined in `inputs/validation_frame_original.json`, and the reference hand solution was taken from `validation_ref/Kim_A7_written.pdf`. The original planar frame problem was embedded into the x-y plane in the present 3D solver by restraining the out-of-plane degrees of freedom. This allowed the 3D implementation to be checked against a known hand solution while also verifying that the solver handled a different plane orientation from the truss validation case.

Figure 3 shows the original frame geometry and loading used in the hand solution, and Figure 4 shows the corresponding computed bending-moment distribution from the 3D solver. The comparison focused on the free-DOF displacement quantities, support reactions, and local element end forces listed in the hand solution.

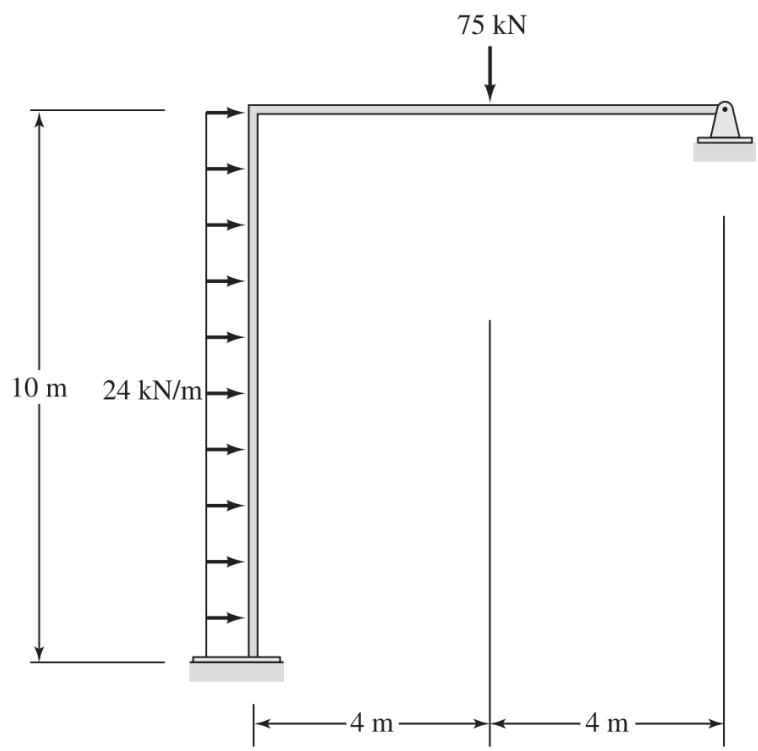

Figure 3: Hand-solution layout, loading, and dimensions for the frame validation problem.

Table 3 compares the free-DOF displacements from the hand calculation and the developed solver. Table 4 compares the support reactions reported at Nodes 1 and 3. Table 5 compares the local element end forces for the two frame members. For the original frame validation case, a sign difference was observed in the local transverse force components q'_3 and q'_9. This difference arises at the stage where local element quantities are defined and transformed, rather than in the global solution itself. In the handwritten 2D frame solution, the local element directions follow the course 2D sign convention. In the present code, however, the frame element is treated within a 3D

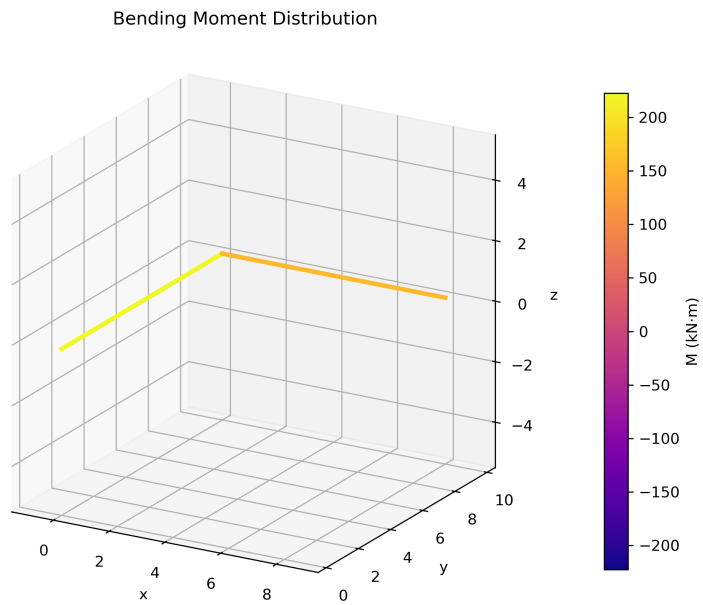

Figure 4: Computed 3D bending-moment distribution for the original frame validation model embedded in the x-y plane.

local-coordinate framework, and the local transverse directions are assigned through the element transformation procedure. As a result, the positive direction of the local transverse shear component can be opposite to that used in the handwritten 2D formulation.

This difference appears specifically in the local end-force recovery step, where the local end-force vector is assembled after transforming the solved displacements into element local coordinates. Because the axial-force and moment components match while only the transverse shear components are sign-reversed, the discrepancy is interpreted as a local sign-convention difference in the element force definition, not as an error in the stiffness formulation or the global displacement solution.

Table 3: Comparison of free-DOF displacements for the original frame validation problem.

Quantity	Hand calculation	Computed result
$u_4 = u_{x2}$	9.5515×10^{-4} m	9.5515×10^{-4} m
$u_5 = u_{y2}$	-5.9949×10^{-4} m	-5.9949×10^{-4} m
$u_6 = \theta_2$	2.5391×10^{-2} rad	2.5391×10^{-2} rad
$u_9 = \theta_3$	2.1201×10^{-2} rad	2.1201×10^{-2} rad

Table 4: Comparison of support reactions for the original frame validation problem.

Quantity	Hand calculation	Computed result
R_{x1}	-126.815 kN	-126.815 kN
R_{y1}	56.831 kN	56.831 kN
M_1	222.802 kN · m	222.802 kN · m
R_{x3}	-113.185 kN	-113.185 kN
R_{y3}	18.169 kN	18.169 kN

Table 5: Comparison of local element end-force components for the original frame validation problem.

Element	Component	Hand calculation	Computed result
1	q'_1	56.831	56.831
1	q'_3	126.815	-126.815
1	q'_5	222.802	222.802
1	q'_7	-56.831	-56.831
1	q'_9	113.185	-113.185
1	q'_{11}	-154.651	-154.651
2	q'_1	113.185	113.185
2	q'_3	56.831	-56.831
2	q'_5	154.651	154.651
2	q'_7	-113.185	-113.185
2	q'_9	18.169	-18.169
2	q'_{11}	0.000	0.000

4.3 Additional Frame Validation Cases

In addition to the original frame problem, three extended validation cases were used to verify the additional features implemented in the solver: prescribed support displacement, temperature loading, and fabrication error. These cases were based on the same frame model used in the original validation, but the right-hand side of the equilibrium equations was modified according to the additional effect considered. The reference hand solutions were taken from `validation_ref/Kim_A10_written.pdf`. In this subsection, the comparison is limited to the free-DOF displacements and support reactions. For more detailed output, including local element quantities and full result tables, the corresponding output files in the `outputs/` directory may be consulted.

4.3.1 Prescribed Support Displacement

The prescribed-displacement case was defined in `inputs/validation_frame_prescribed_displacement.json`. In this case, the top-right pin support was assumed to move 10 mm downward and 5 mm to the right, while the bottom-left support was assumed to undergo a counterclockwise rotation of 0.01 rad. This case was used to verify that imposed support motions were incorporated correctly through the partitioned system equations. Table 6 compares the free-DOF displacements from the hand calculation and the developed solver. Table 7 compares the corresponding support reactions.

Table 6: Comparison of free-DOF displacements for the prescribed-support-displacement frame validation problem.

Quantity	Hand calculation	Computed result
$u_4 = u_{x2}$	0.00593846 m	0.00593846 m
$u_5 = u_{y2}$	−0.00059439 m	−0.00059439 m
$u_6 = \theta_2$	0.02181937 rad	0.02181937 rad
$u_9 = \theta_3$	0.02111055 rad	0.02111055 rad

Table 7: Comparison of support reactions for the prescribed-support-displacement frame validation problem.

Quantity	Hand calculation	Computed result
R_{x1}	−128.79308 kN	−128.79308 kN
R_{y1}	56.34835 kN	56.34835 kN
M_1	238.71761 kN · m	238.71761 kN · m
R_{x3}	−111.20692 kN	−111.20692 kN
R_{y3}	18.65165 kN	18.65165 kN

4.3.2 Temperature Loading

The temperature-loading case was defined in `inputs/validation_frame_temperature_loads.json`. The vertical member was subjected to a uniform temperature increase of +10°C, while the horizontal member was subjected to a temperature gradient with +15°C at the top and +5°C at the bottom. This case was used to verify the thermal fixed-end-force implementation in the solver.

Table 8 compares the free-DOF displacements from the hand calculation and the developed solver. Table 9 compares the corresponding support reactions.

Table 8: Comparison of free-DOF displacements for the temperature-loading frame validation problem.

Quantity	Hand calculation	Computed result
$u_4 = u_{x2}$	-6.1649×10^{-6} m	-6.1649×10^{-6} m
$u_5 = u_{y2}$	6.0199×10^{-4} m	6.0199×10^{-4} m
$u_6 = \theta_2$	2.61669×10^{-2} rad	2.61669×10^{-2} rad
$u_9 = \theta_3$	1.97875×10^{-2} rad	1.97875×10^{-2} rad

4.3.3 Fabrication Error

The fabrication-error case was defined in `inputs/validation_frame_fabrication_error.json`. In this case, both the vertical member and the horizontal member were assumed to be fabricated 3 mm too short. This case was used to verify the implementation of axial fit-up error through equivalent member-level fixed-end-force contributions.

Table 10 compares the free-DOF displacements from the hand calculation and the developed solver.

Table 9: Comparison of support reactions for the temperature-loading frame validation problem.

Quantity	Hand calculation	Computed result
R_{x1}	-126.97054 kN	-126.97054 kN
R_{y1}	56.69115 kN	56.69115 kN
M_1	223.23458 kN · m	223.23458 kN · m
R_{x3}	-113.02946 kN	-113.02946 kN
R_{y3}	18.30885 kN	18.30885 kN

Table 11 compares the corresponding support reactions.

Table 10: Comparison of free-DOF displacements for the fabrication-error frame validation problem.

Quantity	Hand calculation	Computed result
$u_4 = u_{x2}$	0.00395391 m	0.00395391 m
$u_5 = u_{y2}$	-0.00359855 m	-0.00359855 m
$u_6 = \theta_2$	0.02534020 rad	0.02534020 rad
$u_9 = \theta_3$	0.02178841 rad	0.02178841 rad

Table 11: Comparison of support reactions for the fabrication-error frame validation problem.

Quantity	Hand calculation	Computed result
R_{x1}	-126.96129 kN	-126.96129 kN
R_{y1}	56.74281 kN	56.74281 kN
M_1	223.55542 kN · m	223.55542 kN · m
R_{x3}	-113.03871 kN	-113.03871 kN
R_{y3}	18.25719 kN	18.25719 kN

For all three cases, the computed free-DOF displacements and support reactions agree closely with the handwritten reference values. More detailed numerical output, including element-level quantities and auxiliary result sheets, can be found in the corresponding files in the `outputs/` directory.

5. Chaotianmen Bridge Modeling and Baseline Response

5.1 Bridge Selection and Background

The final structure selected for this project is the Chaotianmen Bridge in Chongqing, China. This bridge was chosen because it provides a realistic mixed-element structural system that can be idealized in a meaningful way using both frame and truss elements. The bridge is a steel road–rail arch bridge with a 552 m main span and a total length of 1,741 m, while the main bridge itself consists of three spans of 190 m–552 m–190 m. It carries roadway traffic on the upper deck and roadway and rail transit on the lower deck. At the time of its construction, the 552 m main span was reported as a world-record arch span [1]. More detailed background information on the actual bridge structure, including general dimensions and project data, can be found in the Structurae entry for the Chaotianmen Bridge: `https://structurae.net/en/structures/chaotianmen-bridge`.

From a modeling perspective, this bridge is particularly useful because it combines several types of structural action within one system: arch action in the curved upper members, girder action in the lower chord and deck-related members, and axial-force-dominated behavior in the bracing system. This makes the bridge suitable for a final case study that goes beyond a simple truss or simple frame example and instead demonstrates a bridge-scale mixed truss–frame model.

Figure 5 shows the bridge selected for the final case study, and Figure 6 shows the side-elevation reference drawing used for geometric extraction.

Figure 5: Chaotianmen Bridge in Chongqing, China.

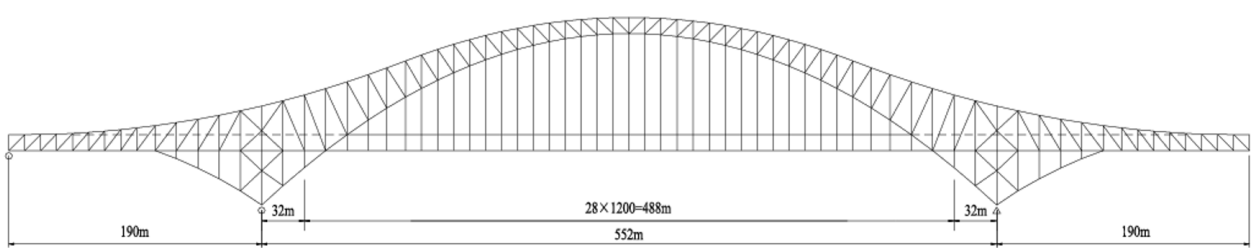

Figure 6: Reference side-elevation drawing used for geometric idealization of the Chaotianmen Bridge model.

5.2 Geometric Input Preparation and 3D Node Generation

A representative side-elevation image of one longitudinal plane was used as the primary geometric reference. Pixel coordinates were sampled from the bridge drawing, and a conversion factor in meters per pixel was established using known dimensions identified from the elevation drawing.

The pixel-coordinate information was obtained using the online image-coordinate tool PixSpy (`https://ko.pixspy.com/`). Based on this conversion, representative nodal coordinates were extracted from the image and organized in a spreadsheet.

In the present model, 195 representative nodes were identified from one longitudinal plane of the bridge. Figure 7 shows the spreadsheet-based extraction of these nodal coordinates after conversion from pixel space to physical dimensions. The extracted coordinates were then assigned structural coordinates in the global x-z plane for one side of the bridge. Additional spreadsheet-based geometric information and conversion calculations are provided in `bridge_info/bridge_calculation.xlsx`.

	Interval	14.485714		conversion	0.9191321	m/px

Bottom node	Pixel_x	Pixel_y	real_x	real_y	Top node	Pixel_x	Pixel_y	real_x	real_y	Top node	Pixel_x	Pixel_y	real_x	real_y
1	4	111	0	0	72	4	97	0	12.86785					
2	18.485714	111	13.314286	0	73	18.485714	97	13.314286	12.86785					
3	32.971429	111	26.628571	0	74	32.971429	97	26.628571	12.86785					
4	47.457143	111	39.942857	0	75	47.457143	97	39.942857	12.86785					
5	61.942857	111	53.257143	0	76	61.942857	96	53.257143	13.786982					
6	76.428571	111	66.571429	0	77	76.428571	95	66.571429	14.706114					
7	90.914286	111	79.885714	0	78	90.914286	94	79.885714	15.625247					
8	105.4	111	93.2	0	79	105.4	93	93.2	16.544379					
9	119.88571	111	106.51429	0	80	119.88571	92	106.51429	17.463511					
10	134.37143	111	119.82857	0	81	134.37143	90	119.82857	19.301775					
11	148.85714	111	133.14286	0	82	148.85714	87	133.14286	22.059172	143	148.85714	117	133.14286	-5.514793
12	163.34286	111	146.45714	0	83	163.34286	85	146.45714	23.897436	144	163.34286	125	146.45714	-12.86785
13	177.82857	111	159.77143	0	84	177.82857	82	159.77143	26.654832	145	177.82857	133	159.77143	-20.22091
14	192.31429	111	173.08571	0	85	192.31429	78	173.08571	30.331361	146	192.31429	143	173.08571	-29.41223
15	206.8	111	186.4	0	86	206.8	74	186.4	34.00789	147	206.8	154	186.4	-39.52268
16	221.28571	111	199.71429	0	87	221.28571	70	199.71429	37.684418	148	221.28571	139	199.71429	-25.7357
17	235.77143	111	213.02857	0	88	235.77143	65	213.02857	42.280079	149	235.77143	124	213.02857	-11.94872
18	250.25714	111	226.34286	0	89	250.25714	60	226.34286	46.87574					
19	264.74286	111	239.65714	0	90	264.74286	54	239.65714	52.390533	150	264.74286	97	239.65714	12.86785
20	279.22857	111	252.97143	0	91	279.22857	49	252.97143	56.986193	151	279.22857	85	252.97143	23.897436
21	293.71429	111	266.28571	0	92	293.71429	43	266.28571	62.500986	152	293.71429	75	266.28571	33.088757
22	308.2	111	279.6	0	93	308.2	38	279.6	67.096647	153	308.2	67	279.6	40.441815
23	322.68571	111	292.91429	0	94	322.68571	33	292.91429	71.692308	154	322.68571	59	292.91429	47.794872
24	337.17143	111	306.22857	0	95	337.17143	28	306.22857	76.287968	155	337.17143	53	306.22857	53.309665
25	351.65714	111	319.54286	0	96	351.65714	24	319.54286	79.964497	156	351.65714	47	319.54286	58.824458
26	366.14286	111	332.85714	0	97	366.14286	20	332.85714	83.641026	157	366.14286	41	332.85714	64.33925
27	380.62857	111	346.17143	0	98	380.62857	17	346.17143	86.398422	158	380.62857	36	346.17143	68.934911
28	395.11429	111	359.48571	0	99	395.11429	14	359.48571	89.155819	159	395.11429	31	359.48571	73.530572
29	409.6	111	372.8	0	100	409.6	11	372.8	91.913215	160	409.6	28	372.8	76.287968
30	424.08571	111	386.11429	0	101	424.08571	9	386.11429	93.751479	161	424.08571	24	386.11429	79.964497
31	438.57143	111	399.42857	0	102	438.57143	7	399.42857	95.589744	162	438.57143	22	399.42857	81.802761
32	453.05714	111	412.74286	0	103	453.05714	5	412.74286	97.428008	163	453.05714	19	412.74286	84.560158
33	467.54286	111	426.05714	0	104	467.54286	4	426.05714	98.34714	164	467.54286	17	426.05714	86.398422
34	482.02857	111	439.37143	0	105	482.02857	3	439.37143	99.266272	165	482.02857	16	439.37143	87.317554
35	496.51429	111	452.68571	0	106	496.51429	2	452.68571	100.1854	166	496.51429	15	452.68571	88.236686
36	511	111	466	0	107	511	2	466	100.1854	167	511	15	466	88.236686
37	525.48571	111	479.31429	0	108	525.48571	2	479.31429	100.1854	168	525.48571	15	479.31429	88.236686
38	539.97143	111	492.62857	0	109	539.97143	3	492.62857	99.266272	169	539.97143	16	492.62857	87.317554
39	554.45714	111	505.94286	0	110	554.45714	4	505.94286	98.34714	170	554.45714	17	505.94286	86.398422
40	568.94286	111	519.25714	0	111	568.94286	5	519.25714	97.428008	171	568.94286	19	519.25714	84.560158
41	583.42857	111	532.57143	0	112	583.42857	7	532.57143	95.589744	172	583.42857	22	532.57143	81.802761
42	597.91429	111	545.88571	0	113	597.91429	9	545.88571	93.751479	173	597.91429	24	545.88571	79.964497
43	612.4	111	559.2	0	114	612.4	11	559.2	91.913215	174	612.4	28	559.2	76.287968
44	626.88571	111	572.51429	0	115	626.88571	14	572.51429	89.155819	175	626.88571	31	572.51429	73.530572
45	641.37143	111	585.82857	0	116	641.37143	17	585.82857	86.398422	176	641.37143	36	585.82857	68.934911
46	655.85714	111	599.14286	0	117	655.85714	20	599.14286	83.641026	177	655.85714	41	599.14286	64.33925
47	670.34286	111	612.45714	0	118	670.34286	24	612.45714	79.964497	178	670.34286	47	612.45714	58.824458

Figure 7: Spreadsheet-based extraction of representative nodal coordinates from the elevation drawing after conversion from pixel coordinates to physical dimensions.

The extracted geometric data were entered into `input_generation.ipynb`, which served as the model-generation notebook for the final bridge study. In this notebook, the nodal information was duplicated in the transverse direction, converted into element connectivity, and combined with the required material, section, support, and load definitions. This workflow was used to generate the bridge input JSON files. The basic input model without any additional special loading cases or scenario-specific modifications was saved as `inputs/chaotianmen_bridge.json`.

5.3 Member Connectivity and Element-Type Definition

The next step was to define the member connectivity and assign an element type to each part of the simplified bridge system. This process, including how the extracted nodes were connected and how each member group was interpreted structurally, was carried out in `input_generation.ipynb`. Using the extracted nodes as geometric reference points, the bridge shape was built by connecting the nodes along the major structural lines visible in the elevation drawing. In this way, the lower chord, the upper top-chord / arch line, the internal vertical and diagonal web system, and the central cable-like region were all defined explicitly from the same set of extracted nodal points.

Figure 8 shows the resulting element-type layout in side elevation.

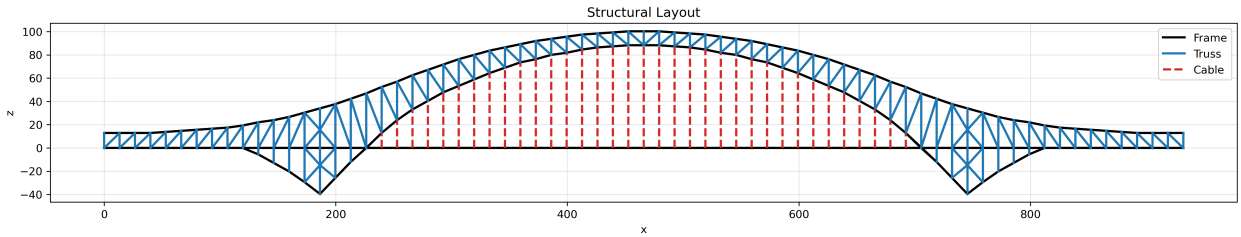

Figure 8: Element-type layout used in the bridge model. Frame members are shown in black, truss members in blue, and cable members in red dashed lines.

The most important modeling choice at this stage was the assignment of frame, truss, and cable elements to different member groups. In the present model, frame elements were used for the bottom chord and the top chord / arch members. These members form the main continuous structural skeleton of the bridge and are expected to resist both axial force and bending. The bottom chord acts as the primary girder-type member running longitudinally through the bridge, while the upper curved chord represents the dominant arch-type load path. Because these members participate directly in the overall stiffness and flexural behavior of the bridge, they were modeled as frame elements rather than as axial-only members.

The vertical and diagonal members were treated differently. Truss elements were assigned to the top verticals, bottom verticals, top diagonals, and bottom diagonals, since these members mainly act as axial-force-transfer components within the web and bracing system. The vertical members connect the upper and lower primary members, while the diagonal members provide the triangulated load path characteristic of a truss-arch structure. In the simplified bridge model, these members were intentionally modeled as truss elements so that their primary axial-force role could be captured without introducing unnecessary local bending behavior.

The central cable-like members visible in the elevation were modeled separately as cable elements. These members were treated as axial-only load-transfer components and were not grouped together with either the continuous frame members or the truss-type web members. This gave a clearer and more rational simplified representation of the different structural roles within the bridge.

Accordingly, the final member grouping used in the model was defined as follows:

- **Frame:** bottom chord and top chord / arch
- **Truss:** top verticals, bottom verticals, top diagonals, and bottom diagonals
- **Cable:** central cable-like members

This grouping is one of the key idealization choices in the final bridge study, because it determines how the extracted nodal layout is converted into an analyzable mixed-element structural model. The detailed connectivity assignment and grouped element-generation logic are documented in `input_generation.ipynb`, where separate parameter values were also assigned to each member group.

A common elastic modulus of $E = 2.0 \times 10^8$ was used throughout the model, while the section properties were differentiated according to structural role. For the bottom chord frame members, the assigned properties were $A = 0.50$, $I = 0.45$, and $J = 0.05$. For the top chord / arch frame members, the assigned properties were $A = 0.40$, $I = 0.30$, and $J = 0.04$. In this way, the two main continuous frame-member groups were given distinct stiffness levels within the simplified bridge model.

For the truss-member groups, separate axial areas were assigned according to member type. The top verticals and bottom verticals were assigned $A = 0.08$, while the top diagonals and bottom diagonals were assigned $A = 0.06$. The cable members were assigned $A = 0.01$, reflecting their intended role as relatively slender axial-only components. Since the truss and cable elements were modeled as axial-force members, only the elastic modulus and axial area were required for these groups.

The three-dimensional bridge representation was completed by duplicating the extracted side-plane nodes across a transverse width of 29.0 m and then introducing transverse members between the two planes. For these transverse members, separate properties were also defined. Transverse frame members were assigned $E = 2.0 \times 10^8$, $A = 0.05$, $I = 0.002$, and $J = 0.001$, while transverse truss members were assigned $E = 2.0 \times 10^8$ and $A = 0.05$. Together, these parameter assignments allowed the model to distinguish among the main longitudinal frame members, the truss-type web and bracing members, the cable-like members, and the transverse connecting members.

5.4 Support and Baseline Load Idealization

After defining the member connectivity and group properties, I assigned the support conditions and the baseline loading case for the bridge model. These were specified at the global structural level so that the simplified bridge could be analyzed as a stable three-dimensional system.

The support conditions were defined at the end and support-related nodes of both longitudinal

planes. In the present model, the restrained nodes were Nodes 1, 71, 147, 187, 196, 266, 342, 382, 192, 193, 194, 195, 387, 388, 389, and 390. At each of these nodes, all six degrees of freedom were restrained: u_x, u_y, u_z, r_x, r_y, and r_z. In other words, these nodes were modeled as fully fixed supports. I adopted this support arrangement to remove rigid-body motion and to keep the simplified bridge model stable during the global analysis. This support configuration should therefore be understood as a modeling assumption for the bridge-scale study rather than as an exact representation of the actual bearing details of the real bridge.

The baseline loading case was defined as the combination of self-weight and representative roadway traffic loading. Self-weight was generated automatically as member loads rather than entered manually at individual nodes. For each member group, the self-weight intensity was determined from

$$w = \gamma A,$$

where A is the assigned section area and γ is the unit weight of steel. Using $\rho \approx 7850$ kg/m^3 and $g = 9.81$ m/s^2, I took

$$\gamma \approx 77.0 \text{ kN/m}^3.$$

As a result, different member groups produced different self-weight contributions according to their assigned cross-sectional areas and member lengths. This allowed the dead load to reflect the relative size of the bottom chord, top chord / arch, web members, cable members, and transverse members within the simplified model.

The roadway traffic load was then introduced as a set of downward nodal loads applied to representative deck nodes. I used this nodal-load idealization to represent the dominant global gravity effect of traffic without attempting to model detailed vehicle positions or wheel-level load transfer. This was sufficient for the purpose of the present study, since the focus here is on the overall bridge response rather than local deck behavior. The resulting baseline loading case, consisting of self-weight together with representative roadway traffic loading, was used as the reference state for the later comparative what-if studies.

5.5 Baseline Response

The baseline response was examined through deformed-shape plots, axial-force plots, shear-force plots, bending-moment plots, support reactions, and tabulated result summaries. These outputs were used to identify the overall deformation pattern, the general force path through the bridge, and the regions of relatively large internal-force demand. The baseline case also served as the reference configuration for the later what-if studies involving altered loading, support movement, temperature effects, fabrication error, and member-level damage scenarios.

6. What-If Scenario Study

After establishing the baseline response, a sequence of what-if scenarios was investigated to examine the sensitivity of the global structural behavior. The goal of these scenarios was not to predict exact field behavior under all circumstances, but to use a set of physically motivated perturbations to identify critical assumptions, sensitive locations, and potential redistribution patterns.

The first group of scenarios involved temperature effects. A reference temperature of 20°C was adopted, and seasonal uniform temperature changes were applied to all structural members to represent winter and summer conditions. Additional extreme hot and extreme cold cases were also considered to extend the thermal sensitivity range. A localized fire case was modeled by applying a large temperature increase only to a selected zone of members near the bridge midspan, so that localized thermal expansion and its effect on force redistribution could be observed. In this simplified fire study, only thermal expansion was considered, while stiffness degradation was neglected.

The second group of scenarios involved fabrication error and support settlement. Fabrication error was modeled as member length error applied to selected sensitive members rather than randomly across the entire bridge. Candidate sensitive members were chosen based on structural role and location, including members near the arch crown, springing region, long diagonals, and transverse closure-type members. Support settlement was modeled through prescribed support displacement, primarily in the vertical direction. Single-support settlement, one-side settlement, and differential settlement cases were considered to investigate how support movement alters global deformation and internal-force redistribution.

The third group of scenarios involved damage and fixity sensitivity. Damage was modeled either by removing selected members from the structural model or by reducing their stiffness to represent degraded load-carrying capacity. Central diagonals, vertical members, and transverse bracing members were natural candidates because their removal can alter the force path of a long-span arch-truss system. In parallel, moment-release scenarios were created by assigning end releases to selected frame elements. These release cases were used both as deterioration-inspired scenarios and as sensitivity studies on the assumed fixity of local structural connections.

Taken together, these scenarios showed how strongly the bridge response depends on both external conditions and modeling assumptions. Some scenarios primarily changed the displacement pattern, while others caused noticeable redistribution of axial force, shear, or bending moment. Temperature, settlement, damage, and release assumptions therefore served not only as isolated load cases, but also as tools for understanding which regions and assumptions govern the global behavior most strongly.

7. Conclusions

This project developed a Python-based direct stiffness method program capable of analyzing mixed 3D truss and 3D frame structural systems from input files alone. The code was organized so that the full workflow, from model definition to result export and plotting, could be reproduced through a consistent repository structure. Additional features such as frame end releases, prescribed support displacements, uniform temperature loading, and fabrication length error were successfully implemented within the same framework.

Validation against hand-solvable benchmark problems confirmed that the truss formulation, frame formulation, and additional features were operating correctly. Embedding planar problems into three-dimensional space provided an effective way to verify the 3D solver while keeping the comparison transparent and rigorous.

The Chaotianmen Bridge case study demonstrated that the developed program can be applied to a realistic mixed-element bridge model. A rational separation between frame members and truss members was established, a two-plane global model was generated, and baseline results were obtained for combined self-weight, road traffic, and rail traffic loading. The resulting figures and tabulated outputs made it possible to identify global deformation characteristics and major internal-force patterns.

Finally, the what-if scenario study showed that the model can be used to investigate a broad range of service, environmental, and accidental conditions. Temperature change, local fire, fabrication error, support settlement, member loss, and moment-release assumptions all provided meaningful insight into the sensitivity of the bridge response. Although the present work remains limited to a simplified linear global analysis, it establishes a strong foundation for future extensions such as nonlinear behavior, buckling-related studies, optimization, stochastic imperfection studies, and more detailed local connection modeling.

8. Proposed Final Structure and Preliminary Background Review

The final structure selected for this project is the Chaotianmen Bridge in Chongqing, China. This bridge is a continuous steel truss arch bridge with tie girders crossing the Yangtze River, with a main span of 552 m and a total length of 1,741 m. It is also a double-deck bridge carrying both roadway and railway traffic.

This structure was selected because it is well suited for the combined use of frame and truss elements. The main chord members of the arch rib, the tie girder, the floorbeams, the stringers, and the major beam members of the deck system can be modeled using frame elements. In contrast, the internal web members of the arch, diagonal bracing members, and some auxiliary connecting members can be idealized as truss elements. This makes the bridge a suitable example of mixed three-dimensional truss–frame analysis.

The preliminary background review focused on three points. First, I examined how similar arch bridges and steel truss arch bridges have been modeled in previous studies. Song et al. analyzed a continuous steel truss arch railway bridge using a three-dimensional numerical model combined with monitoring data [1]. Second, I reviewed how loading conditions, support conditions, and temperature effects have been considered in arch bridge studies. Sun et al. analyzed an arch bridge system including steel truss beams, suspenders, stringers, and lateral beams [2]. Third, I reviewed how local member damage can affect global structural behavior. Fan et al. investigated the influence of hanger damage and arch-beam combination parameters on a through tied-arch bridge [3].

Based on this review, several what-if scenarios will be considered in the final study, including temperature change, support-condition change, asymmetric live loading, and local stiffness reduction or removal of selected truss members or hangers. Overall, the Chaotianmen Bridge is an appropriate final modeling case because of its span scale, structural form, and member configuration that naturally accommodates both truss and frame idealization.

References

[1] Y. sheng Song, Y. liang Ding, H. wei Zhao, G. xin Wang, A. qun Li, and B. ya Cao, "Structural behavior analysis of a continuous steel truss arch railway bridge integrating monitoring data," *Advances in Mechanical Engineering*, vol. 9, no. 1, pp. 1–14, 2017.

[2] N. Sun, X. Zheng, Y. Li, Y. Zhao, H. Yuan, and M. Zhou, "Numerical study on the mechanical performance of a flexible arch composite bridge with steel truss beams over its entire lifespan," *Sustainability*, vol. 16, no. 14, p. 6041, 2024.

[3] B.-H. Fan, Q. Sun, S.-G. Wang, and Q. Chen, "Numerical analysis of failure mechanism in through tied-arch bridges: Impact of hanger damage and arch-beam combination parameters," *Symmetry*, vol. 17, no. 11, p. 1823, 2025.

Appendix

A. Code Organization and Main Functions

This appendix summarizes the organization of the codebase and the main functions used in the project. The program is divided into helper modules so that preprocessing, element formulation, assembly, partitioning, solving, fixed-end force generation, release handling, and postprocessing are kept separate. This structure makes the code easier to debug, validate, and extend.

A.1 `helpers/preprocess.py`

Functions

- `load_model`
- `node_dofs_1based_6dof`
- `restrained_dofs_1based`
- `loaded_dofs_1based`
- `dof_map_truss_3d`
- `dof_map_frame_3d`
- `element_csL`
- `build_global_load_vector`
- `build_global_displacement_vector`

Description The `preprocess` module prepares all model information before assembly and solution. The most important function is `load_model`, which reads a JSON input file and converts the main data structures into a consistent internal format. The module also defines the global DOF numbering convention used throughout the project.

The functions `node_dofs_1based_6dof`, `restrained_dofs_1based`, and `loaded_dofs_1based` translate support and loading information into 1-based global degree-of-freedom indices. The functions `dof_map_truss_3d` and `dof_map_frame_3d` generate the element-to-global DOF mapping for truss and frame elements, respectively. The function `element_csL` computes the direction cosines and element length, which are required later for transformation matrices and load conversion. Finally, `build_global_load_vector` and `build_global_displacement_vector` create the nodal force vector and prescribed-displacement vector from the input file.

A.2 `helpers/elements.py`

Functions

- `get_plane_reference_vector`
- `element_local_axes_3d`
- `truss_transformation_matrix_3d`
- `frame_transformation_matrix_3d`
- `truss_k_local_3d`
- `truss_local_displacements_3d`
- `truss_local_end_forces_3d`
- `truss_axial_force_3d`
- `frame_k_local_3d`
- `frame_local_displacements_3d`
- `frame_local_end_forces_3d`

Description The `elements` module contains the element-level mechanics of the solver. It defines local stiffness matrices, transformation matrices, and local response recovery functions for both 3D truss and 3D frame members.

The functions `get_plane_reference_vector` and `element_local_axes_3d` are used to define the local coordinate system of a 3D member. Based on these axes, `truss_transformation_matrix_3d` and `frame_transformation_matrix_3d` generate the transformation matrices required to move between local and global coordinates.

For truss elements, `truss_k_local_3d` defines the local axial stiffness matrix, `truss_local_displacements_3d` transforms global nodal displacements into local element displacements, `truss_local_end_forces_3d` computes local axial end forces, and `truss_axial_force_3d` extracts the axial member force.

For frame elements, `frame_k_local_3d` defines the local 3D frame stiffness matrix, while `frame_local_displacements` and `frame_local_end_forces_3d` recover the local displacement and internal-force vectors after the global solution is obtained.

A.3 `helpers/assembly.py`

Functions

- `assemble_global_stiffness_and_fef`
- `assemble_global_displacements`
- `assemble_global_forces`

Description The `assembly` module combines element-level quantities into global system quantities. The function `assemble_global_stiffness_and_fef` is the central assembly routine. It loops over all elements, transforms local stiffness matrices to global coordinates, and accumulates both global stiffness contributions and global equivalent fixed-end force vectors.

After the free displacements and restrained reactions are solved, `assemble_global_displacements` reconstructs the complete displacement vector in full global order, and `assemble_global_forces` reconstructs the complete global force vector.

A.4 `helpers/partition.py`

Functions

- `partition_system`

Description The `partition` module separates the global system into free and restrained DOF sets. The function `partition_system` partitions the stiffness matrix, external load vector, prescribed displacements, and equivalent fixed-end force vector into submatrices and subvectors for the standard DSM solution process. This step is essential for handling support conditions, prescribed displacements, and reaction recovery in a consistent way.

A.5 `helpers/solver.py`

Functions

- `solve_free_displacements`
- `compute_reactions`

Description The `solver` module carries out the actual linear-system solution once the partitioned form of the problem is available. The function `solve_free_displacements` evaluates the unknown displacement vector at the free DOFs. The function `compute_reactions` then uses the solved displacements and the restrained DOF information to recover the support reactions. These two functions define the numerical core of the global structural solution stage.

A.6 `helpers/fef.py`

Functions

- `Qf_temp_local`
- `Qf_fab_axial_local`
- `Qf_dist_axial_local`
- `Qf_dist_local_y`
- `Qf_dist_local_z`
- `Qf_dist_local_y_linear`
- `Qf_dist_local_z_linear`
- `Qf_point_local_x`
- `Qf_point_local_y`
- `Qf_point_local_z`
- `Qf_total_local`
- `global_vector_to_local`
- `global_uniform_load_to_local`
- `global_point_load_to_local`
- `global_linear_load_to_local`
- `_get_by_eid`
- `build_Qf_local_from_loads`

Description The `fef` module constructs equivalent local fixed-end force vectors. These are the element-level force contributions associated with temperature loading, fabrication error, distributed member loading, and member point loading.

The functions beginning with `Qf_...` evaluate individual fixed-end force contributions in local coordinates. For example, `Qf_temp_local` is used for thermal effects, `Qf_fab_axial_local` for fabrication length error, and the remaining functions for distributed or point member loads in axial or transverse directions. The conversion functions `global_vector_to_local`, `global_uniform_load_to_local`, `global_point_load_to_local`, and `global_linear_load_to_local` transform user-defined global load data into the local coordinates required by the element formulation.

The most important high-level routine in this module is `build_Qf_local_from_loads`, which collects all member-level load information from the JSON input and assembles the final local fixed-end force vector for each element.

A.7 `helpers/releases.py`

Functions

- `get_release_by_eid`
- `apply_frame_releases_local`
- `add_zero_stiffness_dofs_to_restrained`

Description The `releases` module implements frame member end releases. The function `get_release_by_eid` reads the release definition associated with a particular element. The function `apply_frame_releases_local` modifies the local stiffness matrix and local fixed-end force vector of a frame member to represent the requested release condition. This is the key function for handling moment-release cases in the validation and bridge what-if studies.

The function `add_zero_stiffness_dofs_to_restrained` is used to handle DOFs that become effectively zero-stiffness after release application, preventing singular-system issues by treating such DOFs consistently as restrained in the solution process.

A.8 `helpers/postprocess.py`

Functions

- `print_dsm_results`
- `print_element`

- display_compact
- print_matrix_scaled
- print_vector_scaled
- build_truss_results_dataframe_3d
- build_frame_results_dataframe_3d
- get_node_translation_3d
- get_truss_element_global_displacements_3d
- get_frame_element_global_displacements_3d
- _set_axes_equal_3d
- _get_element_xyz
- _get_node_translation
- _frame_axial_force_from_q_local
- _frame_moment_value_from_q_local
- _frame_shear_value_from_q_local
- _get_scalar_values
- plot_model_3d_plotly
- _plot_colored_structure
- plot_model_undeformed_by_type
- plot_model_undeformed_deformed
- plot_model_axial_force
- plot_model_shear_force
- plot_model_bending_moment
- plot_model_scalar_2d
- plot_model_geometry_2d
- plot_model_deformed_2d
- plot_model_axial_force_2d

- `plot_model_shear_force_2d`
- `plot_model_bending_moment_2d`
- `split_elements_by_y_plane`
- `_get_plane_label`
- `plot_model_geometry_2d_two_planes`
- `plot_model_deformed_2d_two_planes`
- `plot_model_axial_force_2d_two_planes`
- `plot_model_shear_force_2d_two_planes`
- `plot_model_bending_moment_2d_two_planes`
- `plot_results`
- `build_displacement_summary`
- `build_reaction_summary`
- `build_member_force_summary`
- `build_equilibrium_summary`
- `build_release_summary`
- `build_result_summary`
- `build_result_tables`

Description The `postprocess` module is responsible for all result reporting, table generation, and plotting. The functions `print_dsm_results` and `print_element` provide direct console output of global and element-level results. The helper functions `display_compact`, `print_matrix_scaled`, and `print_vector_scaled` are mainly used for readable notebook display and debugging.

The functions `build_truss_results_dataframe_3d` and `build_frame_results_dataframe_3d` collect solved element results into tabular form. These are later used by `build_result_tables`, which exports the spreadsheet files written to the output directory. In addition to the basic response tables, the module also provides summary functions such as `build_displacement_summary`, `build_reaction_summary`, `build_member_force_summary`, `build_equilibrium_summary`, and `build_release_summ` These are used to generate compact engineering summaries of the most important response quantities.

For visualization, the module contains both 3D and 2D plotting routines. The 3D functions generate undeformed geometry, deformed shape, and scalar-response plots for axial force, shear force, and

bending moment. The 2D plotting functions provide simplified side-view representations, while the two-plane plotting functions were added specifically for the Chaotianmen Bridge model so that members at different y-locations can be visualized separately instead of overlapping in a single 2D view. The high-level plotting wrapper is `plot_results`, which manages both display and file-saving options for the generated figures.

A.9 Summary of Workflow

Workflow Steps The overall analysis sequence in the program is as follows:

1. Read the JSON input file with `load_model`.
2. Preprocess the input into DOF maps, support conditions, and load vectors.
3. Build element stiffness matrices and element fixed-end force vectors.
4. Assemble the global stiffness matrix and global equivalent load vector.
5. Partition the system into free and restrained DOFs.
6. Solve for the unknown displacements and recover support reactions.
7. Reconstruct the complete global displacement and force vectors.
8. Compute element-level results.
9. Export tables and generate plots through the postprocessing routines.

This modular sequence allowed the same codebase to be used for validation examples, feature verification, and the final Chaotianmen Bridge case study.